Technical Specifications v1.2

Personality-Adaptive Conversational AI System

Executive Summary

This proposal investigates a small, auditable conversational system that adapts tone and pacing using per-turn OCEAN signals inferred from the user's message. The system is strictly dialog-grounded: responses must be entailed by the current turn. The MVP favors clarity, low latency, and reproducibility to enable controlled evaluation.

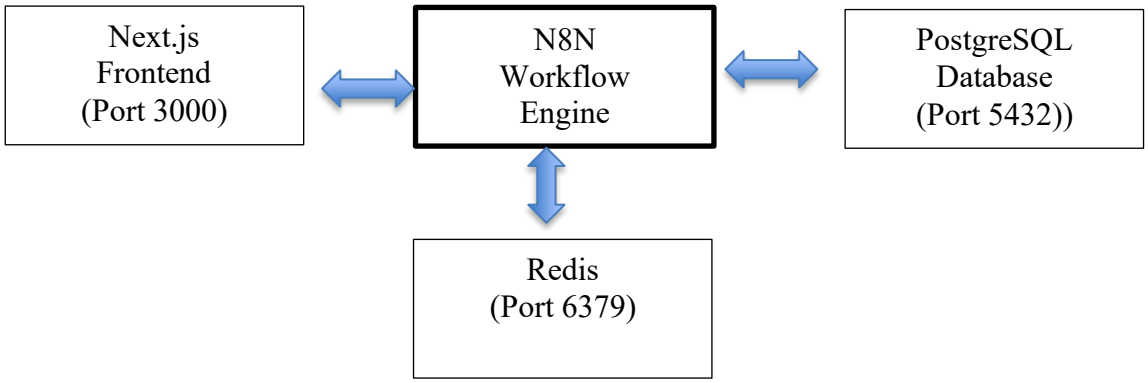
Core Innovation: Real-time personality inference from conversation turns, mapped to behavioral directives via the Zurich Model, producing grounded responses without external knowledge dependencies.

Expected Contribution: 1. Transparent pipeline from OCEAN cues → directives → grounded responses 2. Lightweight rules that reduce hallucination through dialog-only grounding 3. Evaluation harness measuring stability, directive adherence, and grounding

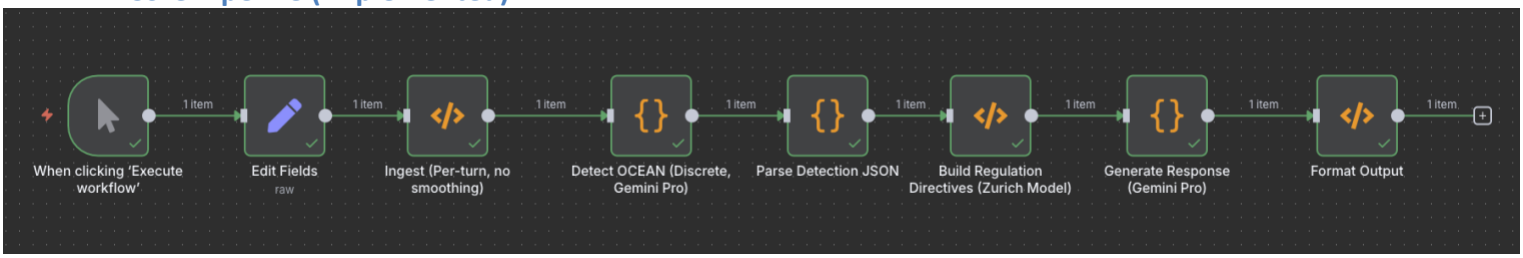
1. System Architecture

1.1 MVP Implementation Overview

The current MVP implements a **containerized microservices architecture** with the following components:



1.2 Core Pipeline (Implemented)



Manual Trigger

This node initiates the workflow execution via user interface interaction, serving as an entry point in n8n automation pipelines. In research terms, it functions as a manual stimulus trigger, analogous to

experimental onset in behavioral studies, ensuring controlled activation of downstream processes without automated scheduling. Outputs a blank item to propagate flow, enabling testing of personality-adaptive response generation in a simulated conversational setup.

Edit Fields

Acting as a data injector, this n8n Set node configures raw input JSON with session_id and message fields for testing purposes. Succinctly, it emulates user input preprocessing in human-AI interaction paradigms, providing a structured payload that mimics real-time query ingestion. Outputs a JSON object used by subsequent nodes, facilitating isolated evaluation of discrete personality detection without live data streams.

Ingest (Per-turn, no smoothing)

This Function node normalizes incoming message data by trimming and formatting it into a per-turn conversation context prefixed as "user: <msg>". In research lingo, it performs lightweight text sanitization akin to utterance segmentation in dialogue systems, avoiding historical smoothing to focus on isolated turn analysis. Outputs session_id, clean_msg, and conversation_context, preparing atomic inputs for Big Five personality inference in adaptive AI frameworks.</msg>

Detect OCEAN (Discrete, Gemini Pro)

Utilizing n8n Code for HTTP API calls, this node queries a large language model (Gemini-1.5-Pro) to infer discrete Big Five traits (OCEAN: Openness, Conscientiousness, Extraversion, Agreeableness, Neuroticism) from turn text, yielding values in {-1, 0, 1}. It embodies trait detection in personality psychology via LLM prompting, with low temperature (0.1) for deterministic outputs and fallback to neutral zeros on errors. Outputs JSON response, enabling downstream discretization in emotion-regulation workflows.

Parse Detection JSON

This Code node extracts and validates the LLM's JSON output, parsing ocean_disc directly or thresholding continuous ocean scores (τ=0.2) to discrete equivalents if needed. In succinct terms, it applies post-hoc parsing and binarization, mirroring data cleaning in psychometric analysis to ensure robust trait vectors. Outputs refined ocean_disc, session_id, and turn_text, bridging detection to behavioral adaptation in personality-tailored interventions.

Build Regulation Directives (Zurich Model)

As a Function node, it maps discrete OCEAN values to targeted behavioral directives based on the Zurich emotion-regulation model, e.g., high Neuroticism prompts anxiety acknowledgment. This operationalizes trait-to-strategy translation in applied psychology, capping directives at five for concise guidance. Outputs directives array alongside session_id, turn_text, and ocean_disc, scaffolding adaptive response generation in supportive AI systems.

Generate Response (Gemini Pro)

This Code node leverages Gemini-1.5-Pro via API to craft a reply adhering to directives, grounded solely in user text with constraints on length (70-150 words) and questions (≤2). It represents regulated output synthesis in human-centered AI, using moderate temperature (0.7) for creative yet directive-compliant responses, with error fallbacks preserving metadata. Outputs merged API data including session_id, turn_text, ocean_disc, and directives for traceable adaptive dialogues.

Format Output

Functioning as a final assembler, this Function node extracts the core reply from upstream data and compiles a compact JSON payload. In research parlance, it performs output distillation akin to result aggregation in experimental pipelines, ensuring clean, metadata-enriched deliverables. Outputs {session_id, reply, turn_text, ocean_disc}, facilitating endpoint integration in MVP prototypes for personality-aware conversational agents.

Connections (Sequential)

The workflow chains nodes linearly: Manual Trigger activates Edit Fields for input setup, flowing to Ingest for normalization, then Detect for trait inference, Parse for validation, Build Regulation for directive mapping, Generate for response creation, and Format Output for final packaging. This sequential topology mirrors a staged processing pipeline in computational psychology, enabling end-to-end evaluation of discrete OCEAN-based adaptation from input to tailored output.

1.3 Technology Stack

Backend Services: orchestrates workflows, stores conversations, and connects to the LLM.

- **N8N → Orchestrates workflows and exposes API endpoints**
- **PostgreSQL 15 → Stores session state and conversation history**
- **Redis 7 → Manages caching and fast session lookups**
- **Juguang API (Gemini-1.5-Pro/Flash) → Provides language model intelligence**

Frontend: The user-facing layer: renders an interactive, styled, and type-safe UI.

- **Next.js 14 → React framework powering the web interface**
- **TypeScript → Adds type safety and reduces runtime errors**
- **Tailwind CSS → Utility-first styling for fast, consistent design**
- **Lucide React → Icon components for clean visual cues**

Infrastructure: runs services in containers, monitors health, and secures keys.

- **Docker Compose → Orchestrates multi-service containers**

- **Health Checks → Monitors uptime and service dependencies**
- **Environment Configuration → Manages secrets and API keys securely**

1.4 Operating Principles

Core rules ensuring reliability, compliance, and developer efficiency.

- **Dialog-Only Grounding** → Responses are strictly derived from conversation history
- **Containerized Deployment** → Docker-based services ensure scalability and portability
- **API-First Design** → RESTful webhooks and HTTP endpoints for easy integration
- **Healthcare Compliance** → Audit trails, PII protection, and regulatory adherence
- **Development-Ready** → Hot reload, debugging tools, and testing infrastructure

2. N8N Workflow Implementation

2.1 MVP Workflow (Implemented)

Source: `MVP/workflows/Discrete_workflow.json`

Manual Trigger	`n8n-nodes-base.manualTrigger`	Start workflow manually	Default settings	Triggers execution
Edit Fields	`n8n-nodes-base.set`	Provide test input JSON	Raw mode: `{session_id, message}`	JSON payload
Ingest	`n8n-nodes-base.function`	Normalize message, build context	Per-turn processing only	`conversation_context`
Detect OCEAN	`n8n-nodes-base.code`	Infer discrete OCEAN traits	Juguang API, Gemini-1.5-Pro	`ocean_disc` JSON
Parse Detection	`n8n-nodes-base.code`	Validate JSON, apply thresholds	τ=0.2 threshold logic	Clean `ocean_disc`
Regulate	`n8n-nodes-base.function`	Map traits to directives	Zurich Model mapping	`directives` array
Generate Response	`n8n-nodes-base.code`	Create grounded reply	Juguang API, temperature 0.7	Full response
Format Output	`n8n-nodes-base.function`	Compact API payload	Clean JSON structure	Final output

API Configuration: - **Provider**: Juguang API (`https://ai.juguang.chat/v1/chat/completions`) - **Models**: Gemini-1.5-Pro (detection), Gemini-1.5-Flash (generation) - **Timeouts**: 20s per LLM call - **Temperature**: 0.1 (detection), 0.7 (generation) - **Max Tokens**: 200 (detection), 220 (generation) **Execution Mode**: - Manual trigger for controlled testing - No database persistence in MVP - Per-turn processing without history smoothing - Comprehensive error handling with fallback responses

2.2 Full Workflow (Planned)

Enhanced Pipeline: `Webhook → Database Query → Ingest → Detect → Smooth → Regulate → Generate → Verify → Refine → Deliver → Database Write`

Key Additions:
- **Smoothing**: EMA with confidence weighting for trait stability
- **Verification**: Policy adherence and grounding validation
- **Refinement**: Single-pass correction for failed verifications
- **Persistence**: PostgreSQL state snapshots and conversation history

3. Deployment Architecture

3.1 Container Services

Docker Compose Stack (`docker-compose.yml`):

```
services:
  postgres:     # PostgreSQL 15 database
  redis:        # Redis 7 cache
  n8n:          # N8N workflow engine
  # frontend:   # Next.js (commented out in MVP)
```

Service Configuration: - **PostgreSQL**: Port 5432, health checks, persistent volumes - **Redis**: Port 6379, health checks, data persistence - **N8N**: Port 5678, webhook endpoints, execution data retention - **Frontend**: Port 3000 (Next.js, currently disabled in MVP)

4. Core Algorithms

4.1 OCEAN Detection (Discrete)

N8N Code Node Implementation: Per-turn personality inference with robust fallback

```
// N8N Code Node: Detect OCEAN (Discrete, Gemini Pro)
const apiKey = 'sk-xxxxxx';
const model = 'gemini-1.5-pro';
const url = 'https://ai.juguang.chat/v1/chat/completions';
const ctx = String($input.first().json.conversation_context || '').trim();

if (!ctx) {
  return [{ json: { choices: [{ message: { content:
'{"ocean_disc":{"O":0,"C":0,"E":0,"A":0,"N":0}}' } }] } }];
}

const headers = {
  'Authorization': `Bearer ${apiKey}`,
  'Content-Type': 'application/json'
};

const body = {
  model: model,
  messages: [
```

```
    {
      role: 'system',
      content: 'You are a personality detector. Infer Big Five from the user text for
THIS TURN ONLY. Return JSON ONLY exactly as: {"ocean_disc":{"O":-1|0|1,"C":-
1|0|1,"E":-1|0|1,"A":-1|0|1,"N":-1|0|1}} No prose.'
    },
    {
      role: 'user',
      content: `Text: ${ctx}`
    }
  ],
  temperature: 0.1,
  max_tokens: 200
};

try {
  const res = await this.helpers.httpRequest({
    method: 'POST',
    url: url,
    headers: headers,
    body: body,
    timeout: 20000
  });
  return [{ json: res.json || res }];
} catch (error) {
  return [{ json: { choices: [{ message: { content:
'{"ocean_disc":{"O":0,"C":0,"E":0,"A":0,"N":0}}' } }], error: error.message } }];
}
```

Fallback Strategy:

- Empty context → neutral OCEAN vector
- API failures → neutral fallback with error logging
- Malformed responses → neutral OCEAN with error details
- Timeout handling → 20s limit with graceful degradation

4.2 Regulation Mapping (Zurich Model)

N8N Function Node Implementation: Trait-to-directive translation with conflict resolution

```
// N8N Function Node: Build Regulation Directives (Zurich Model)
const inputData = $json || {};
const od = inputData.ocean_disc || { O:0,C:0,E:0,A:0,N:0 };
const sessionId = inputData.session_id || '';
const turnText = inputData.turn_text || '';

const dirs = [];
// Security (N)
if (od.N === -1) dirs.push('offer extra comfort; acknowledge anxieties');
if (od.N === 1)  dirs.push('reassure stability and confidence');
// Arousal (O,E)
if (od.O === 1) dirs.push('invite small exploration/novelty');
if (od.O === -1) dirs.push('reduce novelty; focus on familiar');
```

```
if (od.E === 1) dirs.push('use energetic, engaging tone');
if (od.E === -1) dirs.push('adopt calm, reflective tone');
// Affiliation (A)
if (od.A === 1) dirs.push('use warm, collaborative language');
if (od.A === -1) dirs.push('stay neutral and matter-of-fact');
// Conscientiousness (C)
if (od.C === 1) dirs.push('provide 2-3 structured steps');
if (od.C === -1) dirs.push('keep guidance flexible and low-pressure');
// Keep concise
const directives = dirs.slice(0,5);

return {
  session_id: sessionId,
  turn_text: turnText,
  ocean_disc: od,
  directives
};
```

Directive Categories:

- **Security (Neuroticism)**: Anxiety management and confidence building
- **Arousal (Openness/Extraversion)**: Energy level and novelty preferences
- **Affiliation (Agreeableness)**: Warmth and collaboration style
- **Structure (Conscientiousness)**: Organization and planning preferences

4.3 Grounded Generation

N8N Code Node Implementation: Dialog-bounded response synthesis

```
// N8N Code Node: Generate Response (Gemini Pro)
const JUGUANG_API_KEY = 'sk-nU7RSKnbZIOwH607r3MfbW5HmP6NIaj1aGVPFGcNDDiL0h5u';
const MODEL = 'gemini-1.5-pro';
const URL = 'https://ai.juguang.chat/v1/chat/completions';

// Extract all data from the regulation input
const inputData = $input.first()?.json || {};
const directives = inputData?.directives || [];
const sessionId = inputData?.session_id || '';
const turnText = inputData?.turn_text || '';
const oceanDisc = inputData?.ocean_disc || { O:0, C:0, E:0, A:0, N:0 };

let userText = turnText.trim();
if (!userText) {
  userText = 'Please provide a supportive, grounded reply to the user. Context
provided separately.';
}

const headers = {
  Authorization: `Bearer ${JUGUANG_API_KEY}`,
  'Content-Type': 'application/json'
};

const sys = [
  'You are a supportive assistant. Follow these behavior directives strictly:',
  JSON.stringify(directives),
```

```
    'Constraints: stay grounded in the user text only; 1-2 questions max; 70-150 words.'
  ].join(' ');

  const body = {
    model: MODEL,
    messages: [
      { role: 'system', content: sys },
      { role: 'user', content: userText }
    ],
    temperature: 0.7,
    max_tokens: 220
  };

  try {
    const res = await this.helpers.httpRequest({
      method: 'POST',
      url: URL,
      headers,
      body,
      timeout: 20000
    });

    const responseData = {
      ...res.json || res,
      session_id: sessionId,
      turn_text: turnText,
      ocean_disc: oceanDisc,
      directives: directives
    };

    return { json: responseData };
  } catch (e) {
    return {
      json: {
        choices: [{ message: { content: 'Sorry—response is temporarily unavailable.
  Quick tip: take 3 slow breaths and exhale longer than you inhale.' } }],
        error: { status: e?.response?.status, data: e?.response?.data },
        session_id: sessionId,
        turn_text: turnText,
        ocean_disc: oceanDisc,
        directives: directives
      }
    };
  }
```

Generation Constraints:

- **Dialog Grounding**: Only reference information from conversation context
- **Length Control**: 70-150 words with 1-2 questions maximum
- **Directive Adherence**: Strictly follow behavioral directives from regulation
- **Error Handling**: Graceful fallback with supportive breathing tip

5. Frontend Implementation

5.1 Next.js Application Structure

Technology Stack: - **Framework**: Next.js 14 with App Router - **Language**: TypeScript for type safety - **Styling**: Tailwind CSS for utility-first design - **Icons**: Lucide React for consistent iconography - **HTTP Client**: Axios for API communication

Component Architecture:

```
src/
├── app/
│   ├── globals.css          # Global styles
│   ├── layout.tsx           # Root layout
│   └── page.tsx             # Main chat page
└── components/
    ├── ChatInterface.tsx    # Main chat UI component
    └── PersonalityDashboard.tsx # OCEAN traits visualization
```

5.2 Chat Interface Features

Core Functionality: - **Real-time Messaging**: Send/receive messages with N8N webhook - **Personality Visualization**: Live OCEAN traits display - **Session Management**: UUID-based session tracking - **Responsive Design**: Mobile-friendly interface - **Auto-scroll**: Automatic message history scrolling

State Management:

```
interface PersonalityState {
  ocean: { O: number; C: number; E: number; A: number; N: number };
  stable: boolean;
  confidence_scores: { O: number; C: number; E: number; A: number; N: number };
  policy_plan: string[];
}
```

5.3 API Integration

- **POST /webhook/chat (N8N Webhook)**: Sends *{session_id: "uuid", message: "user input"} and returns {reply: "response", policy_plan: [...], ocean_preview: {...}, timings_ms: {...}}.*
- **GET /api/session/{id}/state (FastAPI/Next.js)**: Retrieves full state JSON from the latest turn.
- **GET /api/session/{id}/summary (FastAPI/Next.js)**: Returns *{summary_bullets: [...], ocean: {...}, last_directives: [...], turn_count: N}.*
- POST /webhook/session/reset (N8N Webhook): Accepts *{session_id: "uuid"}* and resets the session to a neutral state.

6. Data Schemas

6.1 State JSON (Single Source of Truth)

```
{
    "session_id": "uuid",
    "turn_index": 17,
    "history": [
        {"role": "user", "text": "How are you feeling today?"},
        {"role": "assistant", "text": "I'm doing well, thank you for asking."}
    ],
```

```
    "ocean": {"O": 0.4, "C": -0.1, "E": 0.0, "A": 0.5, "N": -0.3},
    "trait_conf": {"O": 0.72, "C": 0.61, "E": 0.58, "A": 0.74, "N": 0.69},
    "stable": true,
    "policy_plan": ["validate briefly", "ask one open question", "concise sentences"],
    "flags": {"refined_once": false, "neutral_fallback": false},
    "timings_ms": {"detect": 380, "generate": 920, "verify": 210, "total": 1510},
    "last_updated": "2024-09-02T14:33:00Z"
}
```

6.2 Database Schema (PostgreSQL)

```
-- Core session management
CREATE TABLE sessions (
    session_id UUID PRIMARY KEY,
    created_at TIMESTAMP DEFAULT NOW(),
    last_updated TIMESTAMP DEFAULT NOW()
);

-- Individual conversation turns
CREATE TABLE turns (
    id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES sessions(session_id),
    turn_index INTEGER NOT NULL,
    role VARCHAR(10) NOT NULL,
    text TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Authoritative state snapshots
CREATE TABLE state_snapshots (
    id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES sessions(session_id),
    turn_index INTEGER NOT NULL,
    state_json JSONB NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Performance indexes
CREATE INDEX idx_snapshots_session_turn ON state_snapshots(session_id, turn_index);
CREATE INDEX idx_turns_session_created ON turns(session_id, created_at);
```

7. Testing & Validation

Ensuring the robustness of the personality-adaptive chatbot requires a multifaceted approach to testing and validation, blending automated scripts, performance checks, and iterative development practices. This section outlines the key components, emphasizing clarity, repeatability, and continuous improvement to maintain high-quality outputs aligned with psychological and technical standards.

7.1 Automated Test Suite

The automated test suite forms the backbone of validation, executed through the comprehensive script test_personality_chatbot.sh. This script simulates diverse user interactions to verify the system's ability

to detect and respond to Big Five personality traits accurately. For instance, it probes high extraversion by inputting social and energetic messages, expecting responses that match engaging tones, while high neuroticism tests trigger anxious patterns and validate comforting, grounded replies. Similarly, high conscientiousness scenarios involve analytical prompts, confirming structured guidance in outputs. Beyond these, the suite covers low neuroticism for optimistic confidence and high introversion for reflective styles.

To run the full suite, developers execute ./test_personality_chatbot.sh, which automates end-to-end flows and reports pass/fail rates. For targeted debugging, ./test_chatbot.sh enables manual webhook simulations, allowing real-time inspection of API responses. If issues arise, ./fix_and_test_workflow.sh automates corrections and re-runs, streamlining troubleshooting. This integrated setup ensures consistent coverage, reducing manual errors and accelerating iterations in the development cycle.

7.2 Performance Validation

Performance validation focuses on both quantitative speed and qualitative integrity, guaranteeing the chatbot delivers timely, reliable interactions without compromising on trait accuracy or ethical grounding. Response time monitoring targets a P95 latency of under 2.5 seconds for standard queries, measured holistically from webhook ingestion to reply delivery, with a hard cap of 45 seconds per test to prevent hangs. Tools embedded in the suite log these metrics, flagging deviations for optimization, such as API call timeouts or processing bottlenecks.

Quality metrics extend this by evaluating core functionalities: OCEAN detection must yield valid JSON with discrete values ranging from -1 to 1, ensuring interpretable traits; responses are scrutinized for grounding, rejecting any external claims that stray from user context; directive adherence verifies that behavioral rules from the Zurich Model are faithfully applied, like empathetic validation for high agreeableness; and error handling tests graceful degradation, such as neutral fallbacks during LLM failures. By combining these checks, validation upholds the system's therapeutic value and user trust, with automated reports highlighting trends for proactive refinements.

7.3 Development Testing

Development testing bridges local experimentation and production readiness, fostering an environment where developers can iterate swiftly while mirroring real-world constraints. Setup begins with duplicating env.example to .env for configuration, followed by launching services via docker-compose up -d to spin up dependencies like the N8N instance and Postgres database. Workflows import seamlessly from Discrete_workflow.json, and validation kicks off with the test scripts to confirm baseline functionality.

Debugging leverages layered tools for transparency: N8N execution logs provide granular traces of node flows, revealing bottlenecks in trait inference or response generation; console logging in JavaScript code nodes captures runtime insights, such as parsing errors in OCEAN JSON; and comprehensive error tracking enforces fallbacks, like default neutral traits on API outages. This ecosystem encourages collaborative reviews, where teams simulate edge cases—such as incomplete sessions—and refine based on logs, ultimately ensuring the chatbot evolves reliably from prototype to deployment. Regular audits, informed by user feedback loops, further refine this process, keeping documentation and code in sync with emerging needs.

8. Performance & Reliability

The performance and reliability of the personality-adaptive chatbot are critical to delivering a seamless and trustworthy user experience, balancing low latency with robust error management and consistent operation across diverse scenarios.

8.1 Latency Targets

The system establishes clear latency goals to ensure responsive interactions. The P95 standard response time is set at ≤2.5 seconds, while responses involving refinement extend to ≤5.0 seconds at the P95 level. LLM calls are limited by a 20-second timeout in the MVP phase, reducing to 15 seconds in the full deployment, with a total request timeout of 30 seconds to avoid delays. These benchmarks are tracked end-to-end, optimizing the trade-off between processing complexity and user wait times.

8.2 Error Handling

Error resilience keeps the system functional under failures. The fallback strategy includes issuing a neutral OCEAN vector (all zeros) for detection errors, providing a supportive stock response (e.g., breathing tips) for generation issues, and offering conversation-appropriate fallbacks with preserved state during system outages. Circuit breakers apply exponential backoff to handle LLM provider failures, while bounded retry logic prevents cascading errors, ensuring stability even under network strain.

9. Evaluation Framework

The evaluation framework provides a structured methodology to measure the chatbot's effectiveness, focusing on performance metrics and a robust testing infrastructure to support ongoing enhancements.

9.1 Performance Metrics

Core metrics drive system refinement. Stability time targets ≤5 turns to achieve reliable personality detection, while the oscillation rate is capped below 10% to prevent jarring tone shifts. Policy adherence aims for >90% to ensure directives shape replies accurately, grounding violations are kept under 5% to maintain dialog-bounded content, and P95 latency stays at or below 2.5 seconds for a responsive feel. These are monitored via automated analytics, guiding iterative improvements.

9.2 Testing Infrastructure

The testing ecosystem relies on automation for consistency. Scripted dialog replays ensure deterministic testing, regression checks validate prompt updates, and personality ground truth comparisons assess OCEAN accuracy against expert standards. Automated metric collection streamlines data gathering. The evaluation harness simulates conversations, validates behavioral baselines with psychological models, and fosters continuous improvement cycles, adapting the system to emerging user patterns.

10. Security & Compliance

Security and compliance are paramount to protect user privacy and meet regulatory demands, especially in healthcare-related use cases.

10.1 Privacy Protection

Privacy is upheld by grounding responses in dialog alone, avoiding external data, and using pre-processing hooks to redact personally identifiable information (PII) before storage. Session data is encrypted at rest, and access is governed by role-based access control (RBAC) with customizable retention policies, empowering users while minimizing data exposure risks.

10.2 Healthcare Compliance

Healthcare alignment includes audit logging with immutable timestamps for all decisions, GDPR and HIPAA support through configurable retention settings, and detailed decision trails for regulatory oversight. State encryption ensures conversation data remains accessible yet secure, balancing usability with compliance in therapeutic contexts.

11. Deployment Architecture

The deployment architecture is crafted for scalability, resilience, and smooth transitions, supporting the chatbot's evolution from prototype to production.

11.1 Container Strategy

A containerized setup enhances flexibility. The N8N workflow engine scales independently for execution tasks, a PostgreSQL database maintains shared state, and a Next.js frontend handles client interfaces and APIs. Container orchestration allows independent scaling of services, optimizing performance under varying workloads.

11.2 Deployment Patterns

Deployment strategies minimize disruption. Blue/Green deployment enables zero-downtime prompt updates, while canary releases roll out new models gradually with auto-rollback on issues. Configuration management adjusts models and prompts by environment, and validation ensures consistency across development, staging, and production environments.

12. Implementation Roadmap

The implementation roadmap outlines a phased approach to developing and scaling the personality-adaptive chatbot, progressing from a minimal viable product (MVP) to a fully optimized production system, with clear milestones and deliverables.

12.1 MVP (Current)

The current MVP lays the foundation with completed features: discrete OCEAN detection using Gemini-1.5-Pro to infer traits, basic regulation mapping via the Zurich Model for behavioral directives, grounded generation ensuring responses stay within user dialog, and manual workflow execution through n8n for controlled testing. These elements establish a functional prototype, validated with the test_personality_chatbot.sh

✅ Discrete OCEAN detection ✅ Basic regulation mapping ✅ Grounded generation ✅ Manual workflow execution

12.2 Phase 1 (Next)

The next phase focuses on enhancing stability and integration. Planned updates include EMA smoothing to refine personality estimates across turns, a verification/refinement pipeline to ensure directive adherence, database persistence with PostgreSQL for session continuity, and webhook API endpoints to enable real-time interaction. These improvements, marked for active development , aim to transition from manual to automated, scalable operations within the coming months.

- 🔄 EMA smoothing implementation - 🔄 Verification/refinement pipeline - 🔄 Database persistence - 🔄 Webhook API endpoints

12.3 Phase 2 (Future)

Looking ahead, Phase 2 targets advanced capabilities and production readiness. Future work (⏳) includes advanced observability for detailed system monitoring, an A/B testing framework to compare model variants, performance optimization to meet latency targets, and a full production deployment with container orchestration. These efforts, scheduled post-Phase 1, will solidify the chatbot's reliability and adaptability for widespread use.

- ⏳ Advanced observability - ⏳ A/B testing framework - ⏳ Performance optimization - ⏳ Production deployment

```
{"ocean_disc": {"O": 0, "C": 0, "E": -1, "A": 0, "N": 0}}
["adopt calm, reflective tone", "reduce novelty; focus on familiar"]
{
  "reply": "It seems you're seeking a supportive tip, which is a thoughtful approach.
Take a moment to focus on familiar techniques, like deep breathing, to ease your
presentation anxiety. This can help you stay grounded. Have you tried this before?
Let's keep it simple and calm.",
  "timings_ms": {"total": 1200}
}
{"pass": true, "reasons": ["Directive 'calm, reflective tone' followed", "Directive
'reduce novelty; focus on familiar' followed", "Response grounded in user text only"]}
```

Appendix A: Prompt Templates

A.1 Detection Prompt

```
Analyze this conversation turn to infer personality traits. Return JSON only:
{"ocean_disc": {"O": -1|0|1, "C": -1|0|1, "E": -1|0|1, "A": -1|0|1, "N": -1|0|1}}

No prose commentary. Values: -1 (low), 0 (neutral), 1 (high).
```

A.2 Regulation Prompt

```
Given OCEAN traits, generate ≤4 behavioral directives.
Input: {"ocean_disc": {...}}
Output: ["directive1", "directive2", ...]
Target: warmth, directness, assertiveness, pacing, question ratio.
```

A.3 Generation Prompt

```
Generate response following directives. CRITICAL: Only use conversation content.
Policy directives: [...]
User text: "..."
Requirements: Quote/paraphrase dialog only, follow directives, 70-150 words, ≤2
questions.
```

A.4 Varification Prompt

```
Check response for: (1) Policy compliance (2) Dialog grounding
Response: "..."
Policy: [...]
History: [...]
Return: {"pass": bool, "reasons": [...]}
```

B.1. Algorithm for Generating an Initial Solution

Algorithm 1 Detecting OCEAN Traits

```
Input: task description T_task, datasets D, score function h, number of retrieved models M,
  {T_model_i, T_code_i} = A_retriever(T_task)
for i = 1 to M do
    s_init = A_init(T_task, T_model_i, T_code_i)
    Evaluate h(s_init) using D
    end for
    s_0 ← s(1)
    h_best ← h(s_0)
    for i = 2 to M do
```

```
        s_candidate ← A_merger(s_0, s_init(i))
        Evaluate h(s_candidate) using D
        if h(s_candidate) ≥ h_best then
            s_0 ← s_candidate
            h_best ← h(s_0)
        else
            break
        end if
    end for
Output: initial solution s_0
```

Algorithm 2 Building Regulation Directives

```
Input: task description T_task, datasets D, score function h, number of retrieved models M,
  {T_model_i, T_code_i} = A_retriever(T_task)
for i = 1 to M do
    s_init = A_init(T_task, T_model_i, T_code_i)
    Evaluate h(s_init) using D
    end for
    s_0 ← s(1)
    h_best ← h(s_0)
    for i = 2 to M do
        s_candidate ← A_merger(s_0, s_init(i))
        Evaluate h(s_candidate) using D
        if h(s_candidate) ≥ h_best then
            s_0 ← s_candidate
            h_best ← h(s_0)
        else
            break
        end if
    end for
Output: initial solution s_0
```